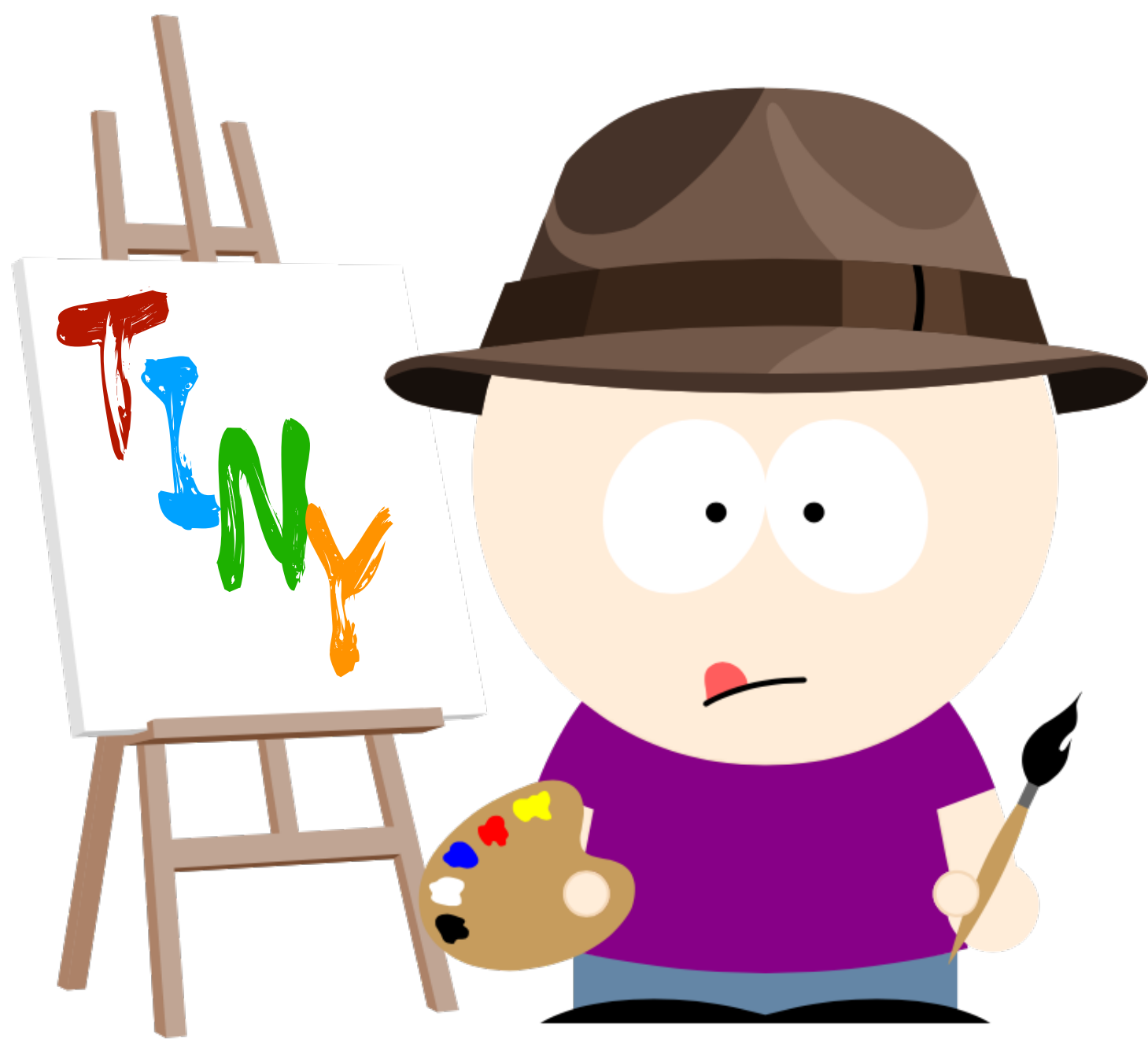




# IN-CLASS EXERCISES



# SEMANTIC ANALYSIS 2

# Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`sem-2-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip sem-2-exercises.zip
```

```
> cd sem-2-exercises
```

```
> ls
```

**TASK 1**

**- THREADED ATTRIBUTES**



# Task 1 (a)

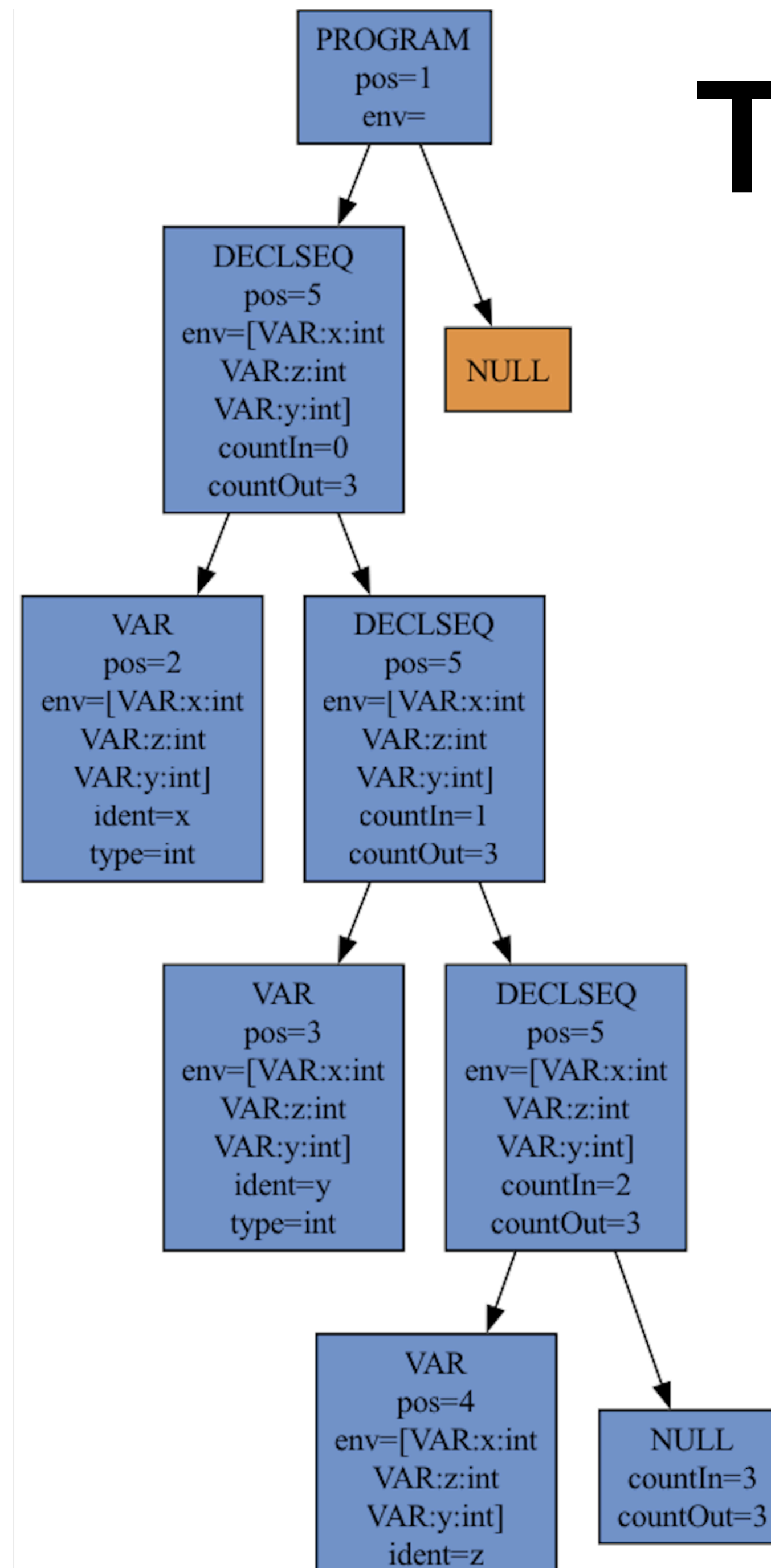
- Your task is to count the number of declarations in a program.
- You should do this with a threaded integer attribute count.

```
BEGIN
    VAR x    : int;
    VAR y    : int;
    VAR z    : int;
END
```

```
> java Sem 8.tny
Number of declarations: 3
```

# Task 1 (b)

- Here's the AST after semantic analysis:
- Check out how the values of `countIn` and `countOut` are passed down the tree (`countIn`) and up the tree (`countOut`)!



# Task 1 (c)

- DECLSEQ and DECLNULL nodes both have the threaded attribute count, i.e. the pair of an inherited attribute countIn and a synthesized attribute countOut:

```
public class DECLSEQ extends AST {  
    public DECL decl;  
    public DECLSEQ next;  
  
    public int countIn;  
    public int countOut;  
}
```

```
public class DECLNULL extends DECLSEQ {  
}
```



# **TASK 2**

## **- TRAVERSING DECLARATIONS**

# Task 2 (a)

- In this task you should walk the declaration sequence in the AST and turn the sequence of symbols into a symbol table.

```
> cat 8.tny
```

```
BEGIN
```

```
    VAR x    : int;
```

```
    VAR y    : int;
```

```
    VAR z    : int;
```

```
END
```

```
> java Sem 8.tny
```

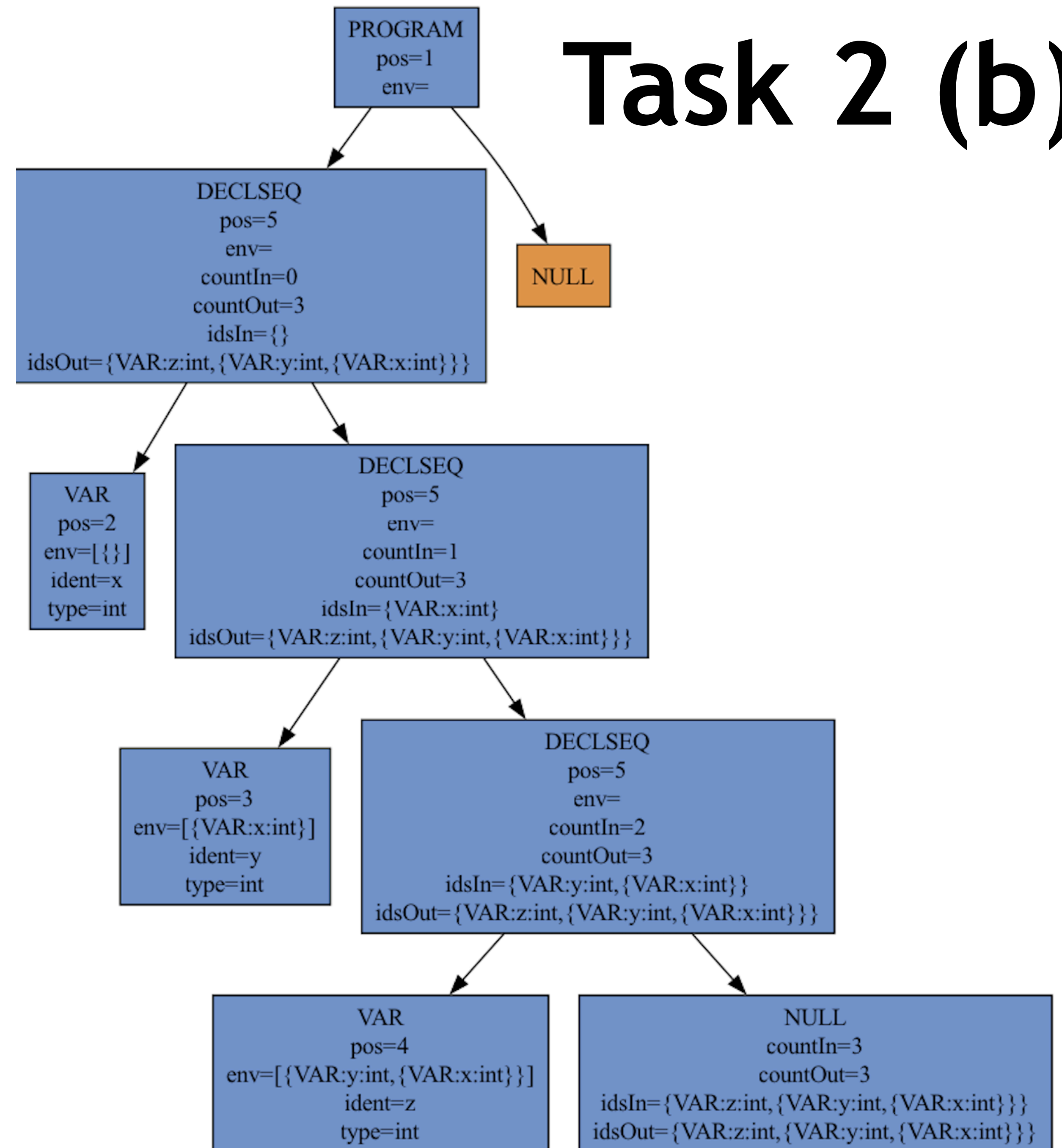
```
Collected symbols: {VAR:z:int,{VAR:y:int,{VAR:x:int}}}
```

```
> dot -Tpng sem_ast.gv > x.png
```



# Task 2 (b)

- Here's the AST after declaration analysis:
- Check out how the values of `idsIn` and `idsOut` are passed down the tree (`idsIn`) and up the tree (`idsOut`)!



# Task 2 (c)

- The declaration sequence has been given a threaded attribute pair, `idsIn` and `idsOut`. They are both of `SyTab` type:

```
public class DECLSEQ extends AST {  
    public DECL decl;  
    public DECLSEQ next;  
  
    public SyTab idsIn;  
    public SyTab idsOut;  
}  
public class DECLNULL extends DECLSEQ {  
}
```

# Symbol Table Class

- On the next page is the interface to the SyTab class.
- Note that it is *immutable*, i.e. it's a list of symbols that you cannot modify, you can only create a new one with a new symbol added:

```
Symbol sy = ...;  
SyTab s = SyTab.create();  
SyTab s1 = s.add(sy);  
Symbol sy1 = s.lookup("x");
```



```
public class SyTab {  
    /* Create a new empty symbol table. */  
    public static SyTab create()  
  
    /* Return a new symbol table with sy at the front. */  
    public SyTab add(Symbol sy)  
  
    /* Look up a symbol by name. */  
    /* Return null if it does not exist. */  
    public Symbol lookup(String ident)  
  
    public String toString()  
}
```

# **TASK 3**

## **- ENVIRONMENTS**

# Task 3 (a)

- Create an environment that only consists of the symbol table you created and pass it down as an inherited attribute to all the statements and expressions:

```
void program(PROGRAM n) {  
    /* BEGIN TASK_3 */  
    Env env = Env.create().cons(n.decls.idsOut);  
    n.stats.env = env;  
    /* END TASK_3 */  
  
    stats(n.stats);  
}
```



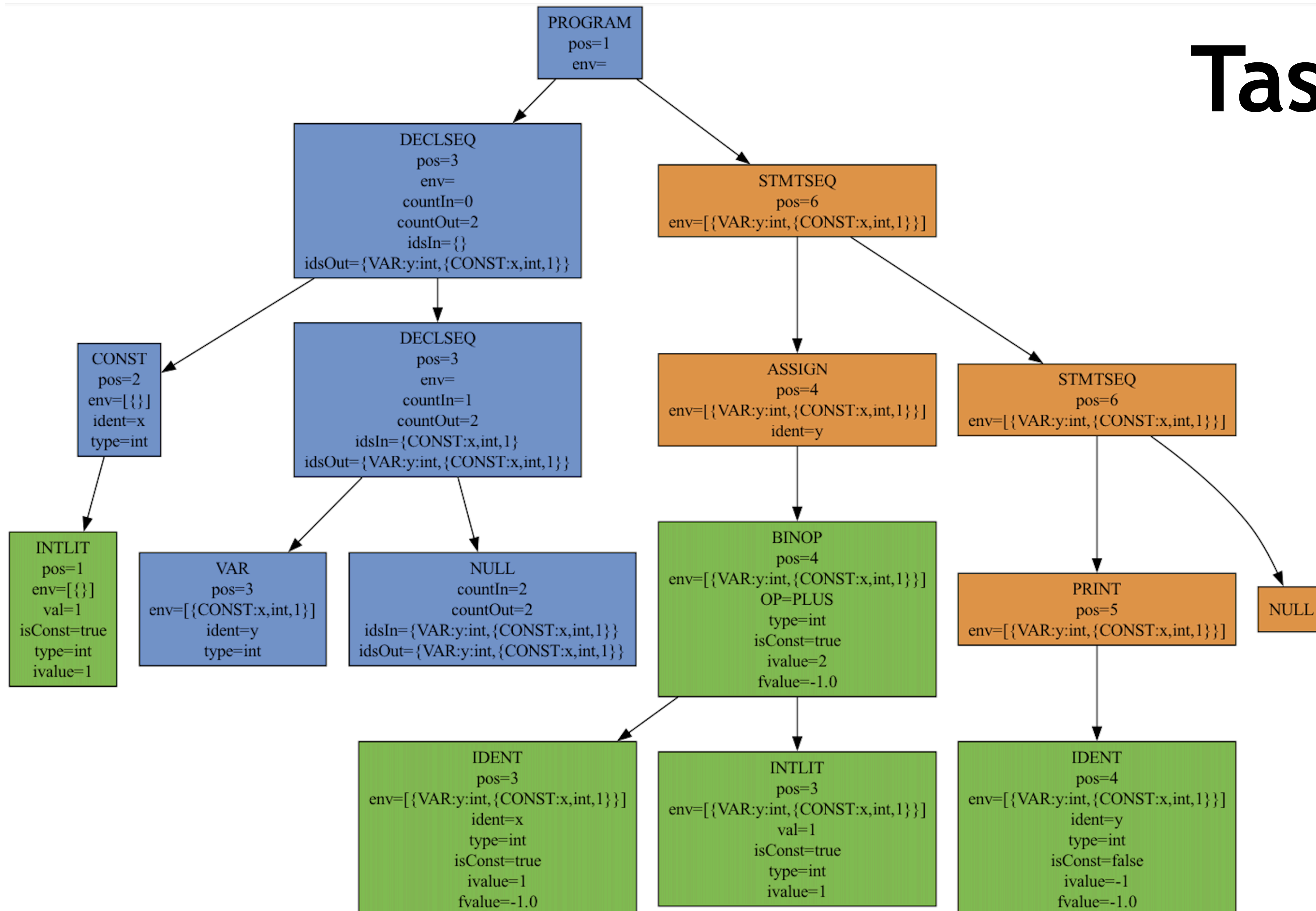
# Task 3(c)

- Here's an example, and the corresponding AST is on the next page:

```
BEGIN
    CONST x : int = 1;
    VAR    y : int;
    y = x + 1;
    PRINT y;
END
```



# Task 3 (b)





# Environment Class

```
public class Env {  
    /* Create a new empty environment.          */  
    public static Env create()  
  
    /* Return a new empty environment with */  
    /* sytab at the front.                */  
    public Env cons(SyTab sytab)  
  
    /* Look up a symbol by name. Return null */  
    /* if it does not exist.                 */  
    public Symbol lookup(String ident)  
  
    public String toString()  
}
```

- Works the same as the SyTab class, but it's a list of SyTabs, not a list of Symbols.

# **TASK 4**

- TRAVERSING**
- STATEMENTS**

# Task 4 (a)

- Imaging that you are create a software engineering tool (like Eclipse, for example), that keeps the program in AST form.
- One of the features you want to support is to give a list of all the variables in the program, and where they are being assigned to:

```
> cat 10.tny
```

```
BEGIN
```

```
    VAR x    : int;
```

```
    x = 42;
```

```
    x = 67;
```

```
END
```

```
> java Sem 10.tny
```

```
x is assigned to on line 3.
```

```
x is assigned to on line 4.
```



# Task 4 (b)

- Add an appropriate *threaded attribute* to STATSEQ and add tree walking code to Sem.java to collect this information.





# DEFINITIONS AND ALGORITHMS

**COLLBERG.CS.ARIZONA.EDU**

**LIGERLABS.ORG**

**SUPPORTED BY**  
**NSF SATC/TTP-1525820**  
**SATC/EDU-2029632**

**COPYRIGHT © 2024-2026**

**CHRISTIAN COLLBERG**

**UNIVERSITY OF ARIZONA**